

COMP25212-EX3-S

Optimisation of Matrix Multiplication

Introduction

This exercise will explore multiplication of two matrices using a computer program, and investigate how the resulting performance is influenced by the underlying computer architecture. It will then investigate strategies to optimise performance, through tailoring of how the matrix multiplication operations are mapped to the underlying hardware. This will include exploiting architectural features such as multiple cores, cache interactions, SIMD, and strategies to avoid hazards.

The task of matrix multiplication is common in scientific computing, as many real-world problems can be modelled as systems of equations which can be grouped together in matrix form. Multiplication of two matrices A and B , resulting in matrix C , is described mathematically as:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} \\ a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} \end{bmatrix} \quad B = \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \\ b_{3,1} & b_{3,2} \\ b_{4,1} & b_{4,2} \end{bmatrix}$$
$$C = AB = \begin{bmatrix} c_{1,1} & c_{1,2} \\ c_{2,1} & c_{2,2} \\ c_{3,1} & c_{3,2} \end{bmatrix} \quad \text{where} \quad \begin{cases} c_{1,1} = a_{1,1}b_{1,1} + a_{1,2}b_{2,1} + a_{1,3}b_{3,1} + a_{1,4}b_{4,1} \\ c_{2,3} = a_{2,1}b_{1,3} + a_{2,2}b_{2,3} + a_{2,3}b_{3,3} + a_{2,4}b_{4,3} \end{cases} \quad (1)$$

Calculation of C requires multiple multiplication and addition operations for each individual term, resulting in a computationally intensive process, particularly for large matrices. The dimensions of the different matrices affect the overall computation, with the convention used here: A (L rows, M cols), B (M rows, N cols), giving C (L rows, N cols).

This work explores the problem where A , B , and C are comprised of double precision floating point values, due to their prevalence in scientific computing, which further increases the computational cost of the process. Each value occupies 64 bits of memory, and arithmetic will be carried out using floating point operations. Overall performance is therefore evaluated as FLOPS – *F*loating *P*oint *O*perations per *S*econd.

For the given matrix multiplication problem, the number of floating point operations (FLOP) can be evaluated as:

$$\text{FLOP} = LN(2M - 1) \quad (2)$$

By monitoring the time taken to complete the multiplication process, we can divide FLOP by the execution time in seconds, to evaluate the number of FLOPS achieved by the hardware for different configurations.

Resources

A base program performing matrix multiplication is provided, along with a build file and script to enable execution of a range of experiments. This code is written in C, and has been prepared to compile with

the `gcc` compiler on Linux. There are no restrictions about the underlying hardware that can be used for this exercise, however there may be certain stages which are not available on particular systems. The code has been tested on the machines in Kilburn 1.10, which contain an Intel Core i7-8700 CPU, and can be booted into Linux.

`main.c`

The provided code performs matrix multiplication, generating two matrices A and B of dimensions: $A(L$ rows, M columns), and $B(M$ rows, N columns), resulting in a matrix C with dimensions (L rows, N columns). After initialising the matrix data structures, a series of matrix multiplication functions are included providing the base functionality from Secs. 1.2-1.7. The program can be compiled using the makefile discussed below, and then executed from the command line using:

```
./matrix_multiply.out <L> <M> <N> <seed>
```

Where L , M , and N define the dimensions of A and B , and `seed` is used to configure the random number generator used to populate A and B with data (double precision values are initialised between 0 and 1). Note that the variable `PRINT_MATRICES` is included to enable printing of the different data structures to the command line: setting this to 1 enables printing, while setting this to 0 causes the application to report only timing data.

`makefile`

The simple `makefile` provided enables compilation of `matrix_multiply.out`, and also creates the assembly language version of the code in `main.s`, which can help in understanding how the specific optimisations are converted into instruction code. The build process can be executed using the ‘make’ command (it’s recommended to run ‘make clean’ beforehand to remove files from the previous build).

`square_mm_profile.sh`

The script file: `square_mm_profile.sh`, is provided to automate execution of the matrix multiplication program on multiple matrices of different dimensions. For simplicity, this assignment will predominantly consider the case of square matrices where $L = M = N$, i.e. where matrices A , B , and C all have dimensions $N \times N$. This script therefore executes the application `matrix_multiply.out` using a range of matrix dimensions, to demonstrate operation. Experiments should be adapted to explore the architecture of the underlying system, e.g. selecting different initial matrix dimensions ($\$N$) and increment sizes ($\dN), and total number of sizes to explore ($\$NUM_TESTS$), to investigate the full range of system performance.

Submission

Submission is via “Canvas” test, however individual ‘Task’ sections are included below, to help guide working and prepare for the “Canvas” submission. For further details on the specific answers and format for individual questions, please check the assignment submission under the *Coursework Exercises* page on “Canvas” .

1 Part I (20 Marks)

1.1 Processor Information (2 mark)

The goal of this assignment is to tailor the matrix multiplication program to the underlying system, in order to optimise execution speed. Submissions will not be assessed on absolute performance, rather on

the evaluation and analysis of how the problem can be optimally mapped to the chosen hardware. It is recommended to use the same hardware throughout the assessment, as this will enable comparison of performance resulting from the optimisations explored in Secs. 1.2-1.7.

Task Describe the hardware being used to run the experiments, and discuss key features relevant to the matrix multiplication application. For example, on Linux use `/proc/cpuinfo`, `lscpu`, or similar to report the specification of the underlying hardware. Discuss which characteristics are likely to be important for the matrix multiplication task, briefly justifying your answers.

1.2 Baseline Performance (3 marks)

The provided code contains the function `matrix_multiply()`, which provides a basic form of matrix multiplication with no effort at optimisation. This function will be used to explore the effect of increasing matrix dimensions, and evaluate baseline performance.

Task Profile the response of multiplying matrices of different sizes using `matrix_multiply()`. Record the: matrix dimensions; total time taken to perform the multiplication; and evaluate the FLOP (floating point operations) involved. You should then use this data to calculate FLOPS (floating point operations per second) achieved by the application/hardware, and plot a graph of FLOPS against matrix dimensions. Discuss how FLOPS changes with matrix size – do variations in L , M and N affect performance in the same way?

Note that the rest of the assignment will consider multiplication of square matrices for simplicity, i.e. the case where $L = M = N$. The script `square_mm_profile.sh` is included to enable automated execution of multiple tests on square matrices. It is suggested to therefore set the experiments to be performed at this stage, such that they can be used as a comparison for all future optimised results.

1.3 Compiler Optimisation (3 marks)

Modern compilers are able to perform many optimisations to improve execution performance. These are typically concerned with making code which runs as fast as possible, or which takes up the least space. For the `gcc` compiler used here, no optimisations are defined in the provided makefile, however the variable `$(OPT)` is defined to enable these to be supplied.

Task Rerun the same experiments from the previous task, exploring the effect of compiler optimisation on performance. Try compiler flags: `$(OPT)=-O`, `-O1`, `-O2`, and `-O3`, by updating the `$(OPT)` variable in the supplied `makefile`. To visualise how performance changes, plot FLOPS vs matrix dimensions for the different configurations explored. Comment on how the compiler achieves the different performance (hint, look in `main.s`). It is recommended to leave the compiler optimisation flag at `-O3` for all remaining experiments, to aid comparison of results.

1.4 Loop Unrolling (3 marks)

The first optimisation to be explored manually is loop unrolling. This technique enables a batch of independent loop iterations to be scheduled in the program without having to go around the loop, thus providing increased numbers of independent instructions for an out of order processor with dynamic scheduling to work with.

Loop unrolling can often be implemented directly by the compiler when loop sizes are known at compile time, however this is difficult for the matrix multiply operations in the given code, as the dimensions

of the loops are specified as runtime arguments. A solution to this is to manually include fixed length loops in the code, such that the compiler can create the optimised program. An example of this manual unrolling is provided in the function: `unrolled_matrix_multiply()`.

Task Familiarise yourself with how the loop unrolling function works, and that it provides the correct answer to the matrix multiplication problem. Test the application in the same manner as previously, and monitor performance for different values of `UNROLL` – note that sticking to powers of 2 is recommended. How does performance compare with that of the previous section? Is this as expected? Are any differences observed in the assembly code for the various configurations?

1.5 Multicore (3 marks)

Many modern processors contain multiple cores, which can work together in tandem on a single application. However, parallel programming is a complex task, and it can be extremely challenging to extract the potential performance from parallel hardware. Parallelisation is implemented here using OpenMP, which is a specification for a set of compiler directives, library routines, and environment variables that can be used to specify high-level parallelism in languages including Fortran and C/C++. A section of code suitable for parallelisation is identified with a pragma; for example a `for` loop suitable for parallelisation can be annotated with:

```
#pragma omp parallel for
```

which can be found inside the function `multicore_matrix_multiply()` in the supplied code. The number of threads to be used is specified using the variable `OMP_THREADS`.

Task Switch the program to use the `multicore_matrix_multiply()` function, and familiarise yourself with its operation (and that it produces the correct numerical result). Rerun the same set of experiments from the previous sections, and record performance for different values of `OMP_THREADS`. How does performance change? Does the underlying system support hyperthreading – if so does adding more threads continue to improve performance?

1.6 Blocking (4 marks)

The importance of the memory hierarchy to program performance has been explored in previous coursework exercises and course lectures. When data is stored and retrieved from within the cache hierarchy, rather than going to main memory, performance can be dramatically improved. The matrix multiplication application is challenging in this respect, as it requires operating on data-structures stored in row-major order, making both row and column based accesses in order to complete the necessary arithmetic. As matrix dimensions grow, it is impossible to fit these data-structures entirely within the cache. One software optimisation which helps overcome this issue is known as *blocking*.

Task A blocked version of the matrix multiplication function `blocked_matrix_multiply()` is provided, with the block size set using the variable `BLOCK_SIZE`. Rerun the same experiments as in the previous sections, exploring the effect of block size on performance. Consider how different block sizes interact with the underlying hardware, and reason why the best block size offers optimal performance.

1.7 SIMD (2 marks)

Many modern processors support SIMD operations: where a Single Instruction is able to operate on Multiple Data. This allows operations to be performed on vectors of operands, with minimal additional

overheads. Sample code is provided utilising x86-64 AVX (Advanced Vector Extensions) supported by Intel (e.g. these should work on any core i-X processor, and may also be directly compatible with AMD processors). These extensions enable floating point operations on 4 64-bit floating point values simultaneously, including loading and storing (256-bit registers are provided to support the vector operations). As these instructions operate on multiple operands within this 256-bit wide word, this approach is also often referred to as *subword parallelism*.

SIMD execution is provided via the function `subword_parallelism_matrix_multiply()`. As it is difficult for compilers to locate sections of code suitable for SIMD operation, C intrinsics are used to specify how the code should be compiled. Four `_mm256` instructions are utilised to complete the load, add, multiply and store operations, with the `_m256d` data type used to hold four double precision variables to be operated on. The loop counters must be updated to account for completing four operations simultaneously: the variable `MM256_STRIDE` is included to help specify this. Note that the data structures must also be 32-byte aligned in memory for the `_mm256` instructions to function correctly. This alignment is specified during the `aligned_alloc()` calls via the variable `MEM_ALIGN`.

Task Switch to the `subword_parallelism_matrix_multiply()` function to perform the matrix multiplication, and familiarise yourself with how the SIMD instructions are implemented. Explore the effect on application performance – is it as expected?

2 Part II (10 Marks)

In Part I, various optimisation techniques were explored independently in isolation. Given the observed performance changes for each technique, explore how combining them can provide further improvements to execution speed on the matrix multiplication problem.

Create your own matrix multiplication function, combining the techniques presented above (and any additional techniques which could help exploit the underlying hardware to improve performance), and report the performance for the same range of tests explored in Part I. Compare the resulting FLOPS measurements with those for the baseline case in Part I, and calculate the overall speed-up achieved through tailoring the application to the underlying hardware.

Take care when combining the methods from Part I, as when operating together this may affect execution, e.g. by changing the minimum matrix dimensions which can be supported.